INDIVIDUAL ASSIGNMENT 1

ML SYSTEM DESIGN DOCUMENT

Design of a Rule-and-AI Hybrid Automated Scoring System for English Speaking and Writing Assessment

Course	DDM501 - AI in DevOps, DataOps, MLOps
Submission	Written Report
Date	24 September 2026

Automating formative English assessment while balancing scoring quality, latency, operational cost, explainability, and human control.

Table of Contents

1. Problem Definition
2. Requirements Analysis
3. Data Requirements
4. Goals and Metrics
5. High-Level Architecture Design
6. MLOps Lifecycle and Continuous Improvement
7. Trade-off Analysis
8. Failure Handling and Graceful Degradation
9. Expected Business Impact
10. Conclusion

1. Problem Definition

1.1 Context and Background

Online English learning platforms can automatically score closed-form activities such as multiple-choice Reading and Listening questions because these tasks have predefined correct answers. Writing and Speaking are more difficult because candidate responses are open-ended and require judgment across multiple linguistic criteria.

In a traditional workflow, essays and speaking recordings are reviewed manually by teachers or examiners. This creates operational limitations as the platform grows: grading workload increases with the number of submissions, feedback can be delayed, and scores may vary between evaluators. A production platform therefore benefits from a scoring service that can provide immediate formative feedback while reserving human attention for difficult or uncertain cases.

The proposed solution is a Rule-and-AI hybrid scoring system for Writing and Speaking. Deterministic rules handle measurable checks such as response length, repetition, structural validation, and invalid input detection. AI components evaluate semantic and linguistic qualities such as task relevance, coherence, grammar, vocabulary, fluency, and feedback quality.

1.2 Problem Statement

The English learning platform requires a scalable method for assessing large volumes of Writing and Speaking submissions without requiring every response to be manually graded. The system should generate estimated overall and criterion-level scores, provide actionable feedback, return results within an acceptable response time, and maintain sufficient agreement with human assessors.

The system is intended for formative learning and practice rather than official high-stakes certification. This scope allows automation to improve feedback speed and accessibility while retaining human review for low-confidence or exceptional cases.

1.3 Current Situation

1. A learner submits a Writing response or Speaking recording.
2. The submission waits for an available teacher or examiner.
3. The evaluator reads or listens to the response, assigns scores, and writes feedback.
4. The result is returned to the learner after the review is completed.

The main limitations are long feedback time, limited scalability, high human workload, and possible inconsistency across evaluators.

1.4 Why Machine Learning is Appropriate

Pure rule-based software is effective for objective checks such as word count, empty responses, duplicated phrases, and format validation, but it cannot adequately judge semantic relevance, coherence, contextual vocabulary use, or the quality of spoken language. AI models are better suited to these unstructured language tasks, while a rule layer keeps deterministic checks transparent and inexpensive.

The hybrid architecture therefore uses conventional software where the problem is deterministic and AI only where semantic interpretation is needed. This reduces unnecessary model calls and

provides clearer failure behavior than an architecture that delegates the complete process to a single generative model.

1.5 Stakeholders

Stakeholder	Primary Goal	Main Concerns
Students	Receive fast, useful assessment feedback	Fairness, understandable feedback, response time
Teachers / Examiners	Reduce repetitive grading workload	Reliability, consistency, ability to review difficult cases
Platform Owner	Scale assessment capability	Cost per assessment, user experience, service growth
Engineering / ML Team	Operate the service reliably	Latency, errors, model behavior, monitoring, versioning
Administrators	Manage scoring configurations	Auditability, rubric and model versions
Data / Privacy Responsible Team	Protect learner data	Retention, access control, voice recordings, privacy

2. Requirements Analysis

2.1 Functional Requirements

Requirement	Description
Writing submission	Accept question, task type, candidate response, and test metadata.
Speaking submission	Accept question, audio recording, and test metadata.
Input validation	Reject empty, invalid, excessively short, or unusable submissions before expensive inference.
Writing scoring	Return overall score, criterion scores, strengths, weaknesses, and improvement suggestions.
Speaking transcription	Convert accepted audio into a transcript before language assessment.
Speaking scoring	Return overall score, criterion scores, transcript, and learner feedback.
Hybrid evaluation	Combine deterministic rule outputs with AI-based semantic evaluation.
Result persistence	Store scores, feedback, processing status, and model/rubric version metadata.
Human review	Route uncertain or failed assessments to a review queue.
Platform integration	Expose scoring capability through a service API independent of the main platform application.

2.2 Writing Scoring Output

- Estimated overall score
- Task response / task achievement score
- Coherence and cohesion score
- Lexical resource score
- Grammar score

- Strengths and weaknesses
- Actionable improvement suggestions

2.3 Speaking Scoring Output

- Estimated overall score
- Fluency score
- Vocabulary score
- Grammar score
- Pronunciation-related indicators
- Speech transcript
- Actionable feedback

2.4 Integration and User Interaction

The scoring subsystem should remain separate from the core application so that it can scale, fail, and be updated independently. The main platform creates a scoring job and receives a job identifier. The scoring worker processes the job asynchronously, writes the result, and the learner interface displays the result when ready.

From the learner perspective, the interaction remains simple: submit the response, view a processing state, and then receive the score and feedback. Internal details such as queueing, model providers, prompt versions, and retries remain hidden.

2.5 Non-Functional Requirements

Category	Metric / Requirement	Proposed Initial Target
Performance	Writing p95 response time	< 15 seconds
Performance	Speaking p95 response time	< 30 seconds
API	Initial acknowledgement	< 500 ms
Reliability	Service availability	>= 99.5%
Reliability	Successful scoring jobs	>= 99%
Scalability	Daily volume	10,000 scoring requests/day
Scalability	Concurrent jobs	100 concurrent scoring jobs
Maintainability	Versioning	Store model, prompt, rubric, and scoring versions
Maintainability	Monitoring	Operational, cost, and quality metrics

These values are design targets for the proposed system and can be recalibrated after production measurements are available.

3. Data Requirements

3.1 Data Sources

- Writing responses and associated questions
- Speaking audio recordings
- Speech-to-text transcripts
- Human overall scores and criterion-level scores
- Rubric definitions

- Historical AI scoring results and system metadata

3.2 Data Quality

Data Type	Typical Quality Issues	Handling Strategy
Writing	Very short answer, copied text, off-topic response, malformed text	Validation rules, filtering, flags for review
Speaking	Background noise, weak microphone, silence, incomplete recording	Audio validation, retry/re-record flow
Transcript	Speech recognition errors, accent sensitivity	Store transcript quality signals and route uncertain cases
Human labels	Inter-rater disagreement, inconsistent criteria	Use multiple reference scores where possible and monitor agreement

3.3 Privacy and Security

Speaking audio may contain identifiable voice information and assessment results are user-related educational data. The system should therefore minimize retained data, separate identity metadata from evaluation data where possible, encrypt data in transit and at rest, enforce access control, maintain audit logs, and define retention periods for audio and model-evaluation datasets.

3.4 Initial Data Volume Assumptions

A realistic initial evaluation corpus may contain approximately 5,000-20,000 Writing responses and 2,000-10,000 Speaking recordings. The exact quantity is less important than label quality: a smaller set of reliably human-scored examples can be more useful for calibration than a much larger noisy dataset.

4. Goals and Metrics

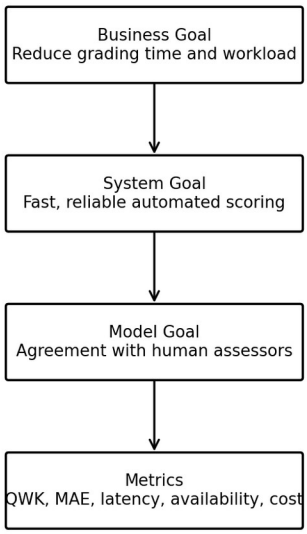


Figure 1. Goal hierarchy from business outcomes to measurable model and system metrics.

4.1 Business Goals

Business Goal	Metric	Proposed Target
Reduce manual grading workload	Percentage of responses requiring human review	Reduce by 60-70% for practice assessments
Reduce feedback waiting time	Median / p95 time to result	Move from hours or days to seconds or minutes
Control assessment cost	Infrastructure and AI cost per successful submission	Continuously measured and optimized
Increase service availability	Hours when scoring is accessible	24/7 automated practice capability

4.2 System Goals

Goal	Metric	Threshold
Fast Writing assessment	p95 latency	< 15 s
Fast Speaking assessment	p95 latency	< 30 s
Reliable service	Availability	>= 99.5%
Reliable job completion	Successful job rate	>= 99%
Scalable processing	Queue delay and throughput	Within SLA at target load

4.3 Model Goals

The central model goal is agreement with expert human assessment rather than simple exact-match accuracy. English proficiency scores are ordered values: predicting 6.0 when the human score is 6.5 is a smaller error than predicting 4.0. Therefore ordinal agreement metrics are more informative than standard classification accuracy.

Metric	Purpose	Proposed Initial Target
Quadratic Weighted Cohen's Kappa (QWK)	Measures agreement while penalizing larger disagreements more strongly	>= 0.75
Mean Absolute Error (MAE)	Average absolute difference between AI and human scores	<= 0.5 score band
Pearson / Spearman correlation	Additional relationship measure between AI and human scores	Track during evaluation
Criterion-level error	Detects systematic weakness in grammar, vocabulary, fluency, etc.	No criterion significantly worse than overall target

4.4 Baseline

A simple baseline is a single AI model call with a fixed rubric prompt that directly returns a score and feedback. The proposed hybrid architecture should only be preferred if it provides measurable benefits such as better consistency, lower cost, improved robustness, easier debugging, or clearer operational control.

5. High-Level Architecture Design

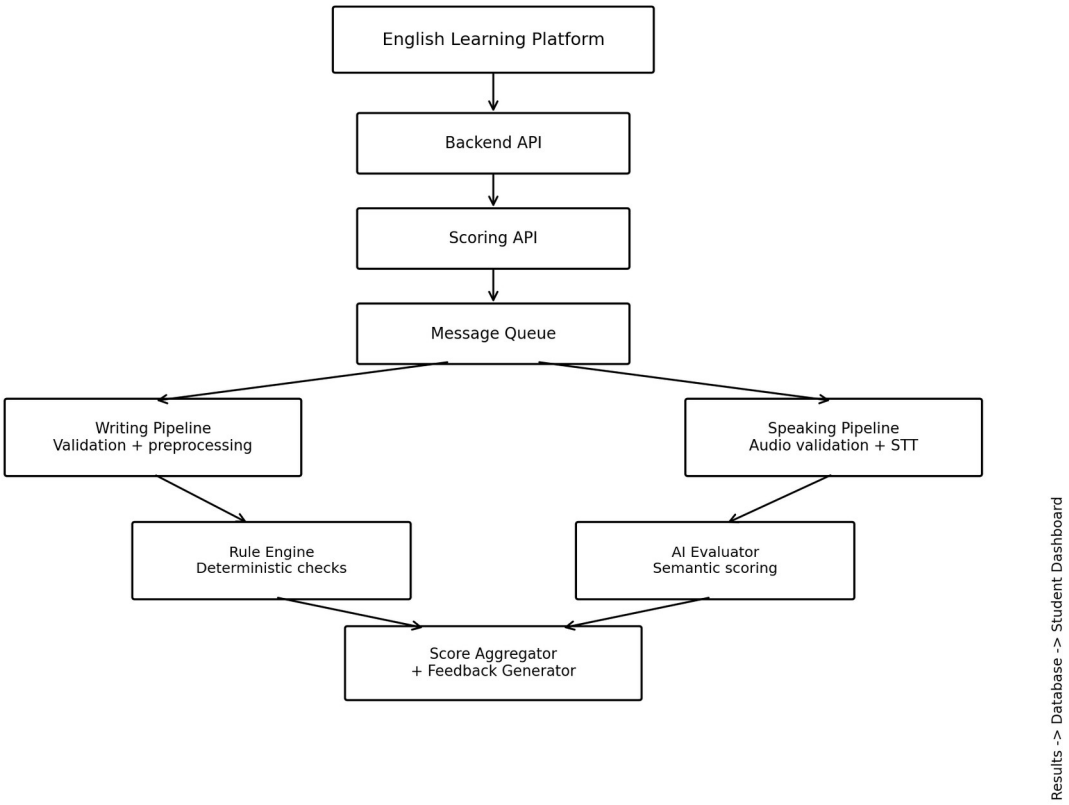


Figure 2. High-level architecture of the Rule-and-AI hybrid English scoring system.

5.1 Main Components

Component	Purpose / Responsibility	Technology Considerations
English Platform	Hosts tests and displays results	Existing web application
Backend API	Authentication, test workflow, job creation	Existing application backend
Scoring API	Entry point to the ML subsystem	REST API; independently deployable service
Message Queue	Buffers scoring jobs and absorbs traffic spikes	Managed queue or broker
Writing Worker	Validates and processes essays	Stateless worker; horizontally scalable
Speaking Worker	Validates audio and performs STT before scoring	CPU/GPU or managed STT service
Rule Engine	Deterministic checks and measurable features	Conventional application logic
AI Evaluator	Semantic rubric-based evaluation	LLM or domain model behind a provider abstraction
Score Aggregator	Combines rule and AI outputs	Versioned business logic
Result Store	Persists scores, feedback, versions, and status	Relational database
Object Storage	Stores audio when retention is allowed	Encrypted object storage
Monitoring	Tracks operations, cost, and quality	Metrics, logs, dashboards, alerts

5.2 Writing Pipeline

5. Receive the essay and task metadata.
6. Validate minimum length, language, duplicate content, and malformed input.
7. Extract deterministic features and rule-based signals.
8. Send the response and rubric to the AI evaluator using a structured output schema.
9. Combine rule-based signals with the AI evaluation using a versioned score aggregator.
10. Validate the final output, persist it, and publish the result to the learner interface.

5.3 Speaking Pipeline

11. Receive and validate the audio recording.
12. Perform speech-to-text transcription.
13. Capture transcript and available speech-related signals.
14. Run rule-based validation for duration, silence, invalid input, and transcript usability.
15. Evaluate transcript and speech information against the speaking rubric.
16. Aggregate scores, generate feedback, persist the result, and return it to the learner.

5.4 Asynchronous Processing

Long-running scoring should not block the main web request. The backend creates a job, writes it to a queue, and returns a job identifier. Workers process jobs independently, allowing retries and horizontal scaling. During load spikes, requests remain queued rather than overwhelming the AI service. This also isolates the main learning platform from transient scoring failures.

5.5 Structured Outputs and Versioning

AI outputs should follow a strict schema containing overall score, criterion scores, explanations, and feedback. The scoring service validates that schema before storing or displaying the result. Each assessment should record model_version, prompt_version, rubric_version, score_version, and timestamp so that historical results can be audited and compared after future system changes.

6. MLOps Lifecycle and Continuous Improvement

Versioned Artifact	**Purpose**
Model version	Identifies the AI/ML model used for scoring.
Prompt version	Tracks instructions and structured scoring prompts.
Rubric version	Tracks the assessment criteria and scoring interpretation.
Rule-set version	Tracks deterministic validation and feature rules.
STT version	Tracks the speech-to-text component used for Speaking.
Aggregator version	Tracks how rule and AI signals are combined.
Monitoring Layer	**Example Metrics**
Infrastructure	CPU/GPU utilization, memory, queue depth, worker health
Application / System	p50/p95/p99 latency, throughput, timeout rate, availability, retry rate
ML Quality	QWK, MAE, score distribution, AI-human disagreement, criterion-level error, input drift
Business	Cost per assessment, manual-review rate, feedback turnaround time, completion rate
Trigger Evidence	**Example Response**

AI-human QWK falls below the accepted threshold	Create a quality alert and run offline re-evaluation on recent human-reviewed samples.
MAE or criterion-level error increases	Inspect which criteria and input groups are affected before recalibration.
Input distribution shifts materially	Sample new data and verify whether performance also changed.
Invalid AI output or timeout rate increases	Investigate prompt/schema/provider changes and operational failures.
Cost per assessment exceeds target	Review token usage, model choice, call count, and caching opportunities.

The MLOps lifecycle separates online production scoring, operational incident handling, ML-quality evidence collection, human validation, candidate improvement, evaluation, and controlled release. Prediction and operational logs always feed production monitoring; they are not decision nodes themselves. Monitoring then evaluates operational health and ML quality through separate decision paths. If no issue or quality/drift signal is detected, the system continues production and monitoring without changing the scoring configuration. Quality/drift signals initiate sample selection and human review, while operational incidents are handled through engineering investigation, fixes, or rollback.

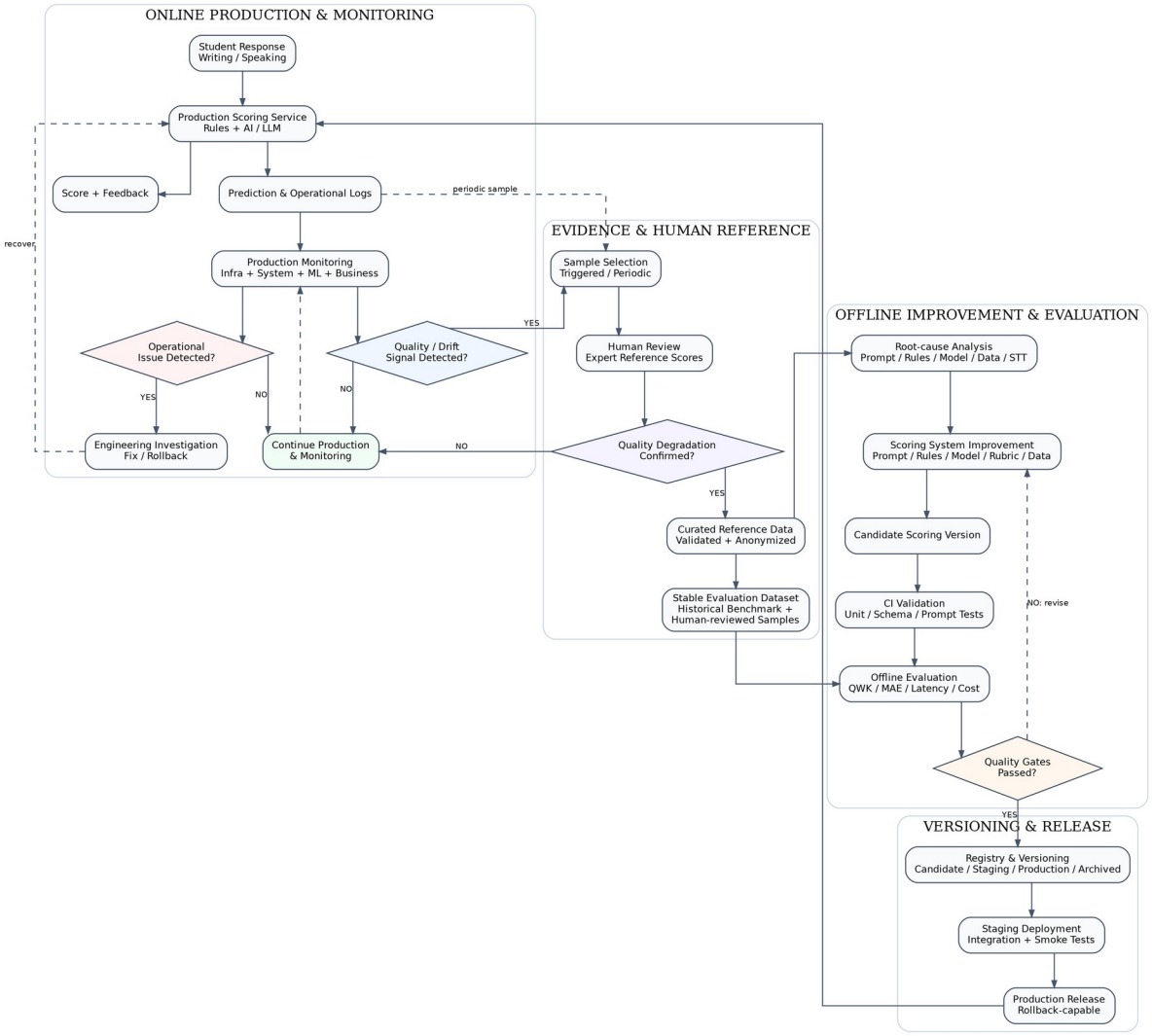


Figure 3. MLOps lifecycle with explicit operational and ML-quality decision paths, human evidence confirmation, offline evaluation, and controlled release.

6.1 Data and Feedback Pipeline

Production Writing responses, Speaking recordings, transcripts, scores, feedback, and operational logs provide evidence about how the scoring service behaves in practice. The production data flow and the evaluation data flow are intentionally separated. Raw production records are not automatically treated as training or evaluation data. Selected samples must pass validation, privacy controls, and anonymization before they are used for human review or model-quality assessment.

- Production logs capture inputs, outputs, model/configuration versions, latency, errors, and selected scoring metadata.
- Triggered or periodic sampling selects representative cases for quality review.
- Human examiners provide trusted reference scores for selected cases.
- Validated human-reviewed samples can be added to the curated reference pool and, when appropriate, to the stable evaluation dataset.

6.2 Scoring Configuration Versioning

A production score can change because of several components, not only the base AI model. Each scoring result should therefore be traceable to a versioned scoring configuration.

- AI model version
- Prompt version
- Rule-engine version
- Rubric version
- Speech-to-text version for Speaking

Version metadata improves reproducibility and auditability. If the same response receives a different score after an update, the engineering team can identify which scoring configuration produced each result.

6.3 Registry and Artifact Management

Approved configurations should be stored in a registry together with evaluation evidence and lifecycle status. Candidate versions are first validated and evaluated; only versions that pass quality gates are promoted as approved artifacts for Staging -> Production -> Archived lifecycle states. Registry metadata should include the configuration version, evaluation date, QWK, MAE, latency, cost, known limitations, and deployment status. The report treats the registry conceptually; a specific implementation such as MLflow is optional.

6.4 Continuous Integration and Evaluation

Changes to scoring code, prompts, rules, rubrics, model settings, or speech-processing configuration should pass automated checks before deployment. The release pipeline combines conventional software tests with ML-specific offline evaluation.

1. Create a candidate scoring version from the proposed improvement.

2. Run unit, schema, prompt, and integration tests.

3. Evaluate the candidate against the stable human-scored evaluation dataset.

4. Measure QWK, MAE, latency, cost, error rate, and output-schema validity.

5. Reject or revise the candidate when any quality gate fails; only approved candidates continue to release.

6.5 Deployment Pipeline and CI/CD

A candidate that passes CI checks, offline evaluation, and quality gates is registered as an approved version and promoted to staging before production. Staging is used for integration tests, smoke tests, output-schema validation, and compatibility checks with the English platform. Production deployment should support rollback to the previous stable scoring configuration if operational or scoring regressions are detected. This controlled release model is preferred over automatic deployment because changes to an ML scoring system require evidence that quality has not degraded.

6.6 Production Monitoring

Monitoring operates at four levels so that infrastructure problems, application failures, ML-quality degradation, and business impact can be distinguished.

- Infrastructure: CPU/GPU utilization, memory, worker health, and queue depth.
- System: p50/p95/p99 latency, throughput, timeout rate, error rate, successful job rate, and availability.
- ML quality: score distribution, QWK/MAE on reviewed samples, criterion-level error, input drift, and AI-human disagreement.
- Business: cost per assessment, manual-review rate, feedback turnaround time, and percentage of successfully automated assessments.

6.7 Evidence-Based Trigger Mechanism

Prediction and operational logs feed a common monitoring layer, after which two independent decisions are evaluated. Operational issues such as latency, errors, timeouts, queue backlog, or provider failures follow an engineering investigation and fix/rollback path. ML-quality or drift signals follow a separate evidence path. If no operational issue and no quality/drift signal is detected, the production scoring configuration remains unchanged and monitoring continues. A detected quality/drift signal initiates sample selection and human review rather than directly triggering retraining or model modification.

Example quality-trigger conditions include:

- AI-human QWK falls below the accepted threshold on a recent reviewed sample.
- MAE or criterion-level error increases beyond the accepted range.
- Input distributions such as response length, audio duration, or language/accent mix shift materially.

- Manual-review rate increases because of low-confidence or inconsistent outputs.

After human review, a second decision confirms whether scoring-quality degradation is supported by expert evidence. If degradation is not confirmed, the system returns to normal production monitoring. If it is confirmed, the reviewed cases become curated reference data, root-cause analysis is performed, and an improvement candidate may be created. This prevents false-positive drift signals from causing unnecessary model, prompt, rule, or rubric changes.

6.8 Human-in-the-Loop and Reference Data

Human review has two roles. First, difficult or high-disagreement production cases can be reviewed to confirm whether an observed quality/drift signal represents real scoring degradation. Second, confirmed reviewed cases provide trusted reference labels for diagnosis and later evaluation. If degradation is not confirmed, the workflow returns to continued monitoring without changing the current scoring configuration. If degradation is confirmed, validated and anonymized cases feed the curated reference-data pool. The stable evaluation dataset is built from historical benchmark cases plus carefully selected human-reviewed samples and is used to evaluate future candidate versions.

6.9 Continuous Improvement and Rollback

Root-cause analysis determines which component should change. The appropriate improvement may be a prompt revision, rule adjustment, rubric clarification, speech-to-text configuration change, model replacement or fine-tuning, preprocessing change, or dataset improvement. The proposed change becomes a new candidate scoring version. The candidate is then evaluated against the stable evaluation dataset and must pass quality gates before registry promotion and deployment. If production monitoring later shows a regression, the system should roll back to the previous stable version while the new version is investigated.

7. Trade-off Analysis

Trade-off	Design Tension	Decision
Model quality vs. latency	Larger or multi-step models may improve language judgment but increase response time.	Use a model sufficiently capable for rubric reasoning while keeping deterministic tasks out of the AI path.
Rule-based vs. AI evaluation	Rules are cheap and predictable but weak on semantics; AI is flexible but nondeterministic and costly.	Adopt a hybrid architecture: rules for deterministic signals, AI for semantic judgment.
Automation vs. human control	Full automation scales well but may mishandle unusual responses; full human review does not scale.	Automatically score normal cases and route uncertain cases to human review.
Quality vs. cost	Multiple specialized model calls may improve feedback but increase token usage, latency, and failure points.	Prefer structured multi-criterion evaluation and split calls only when offline tests show measurable value.
Privacy vs. model improvement	Retaining essays and audio improves evaluation datasets but increases privacy exposure.	Minimize retention, anonymize evaluation data, and keep identity metadata separate where possible.

7.1 Key Architectural Decision

The central design decision is not to treat the LLM as the complete system. The AI model is one component inside a controlled scoring pipeline. Validation, deterministic checks, versioning, monitoring, human review, failure handling, and business metrics remain first-class system components.

8. Failure Handling and Graceful Degradation

Failure Scenario	Expected Behavior
Invalid Writing input	Reject early with a clear validation message; do not invoke AI.
Unusable or empty audio	Ask the learner to record again.
Speech-to-text timeout	Retry according to policy; then queue for reprocessing or review.
AI provider timeout	Retry with bounded attempts; avoid blocking the main platform.
Malformed AI output	Reject schema-invalid output and retry or route to review.
Low-confidence result	Do not treat it as equally trustworthy; send to human review.
Monitoring degradation	Alert operations and preserve rollback to a known version.

Graceful degradation is important because the scoring subsystem should not make the whole English learning platform unavailable. When scoring is temporarily delayed, the platform can still accept the assessment and show a pending status until processing completes.

9. Expected Business Impact

KPI	Expected Direction / Target
Manual grading workload	Reduce by approximately 60-70% for practice assessments
Feedback waiting time	Reduce from hours/days to seconds/minutes
Scoring availability	Provide 24/7 automated practice feedback
Successful scoring jobs	Maintain >= 99%
AI-human agreement	Target QWK >= 0.75 on evaluation data
Average scoring error	Target MAE <= 0.5 score band
Cost per successful assessment	Measure continuously and reduce through architecture/model optimization

Business success should not be inferred from model quality alone. A model may show strong agreement with human scores but still be unsuitable if latency, availability, cost, or user experience are poor. The system therefore evaluates success at business, system, and model levels simultaneously.

10. Conclusion

This design proposes a production-oriented Rule-and-AI hybrid system for automated English Writing and Speaking assessment. The architecture combines deterministic validation and rule-

based signals with AI-based semantic evaluation, asynchronous job processing, structured outputs, monitoring, versioning, human review, and continuous evaluation.

The approach is intentionally broader than a single model. It treats automated scoring as an ML system whose value depends on the complete lifecycle: data quality, model behavior, reliability, scalability, cost, privacy, monitoring, and safe operational control.

The proposed system is designed for formative learning. Its purpose is to provide learners with fast and consistent practice feedback at scale while keeping humans involved in cases where confidence, data quality, or model behavior is insufficient for reliable automation.